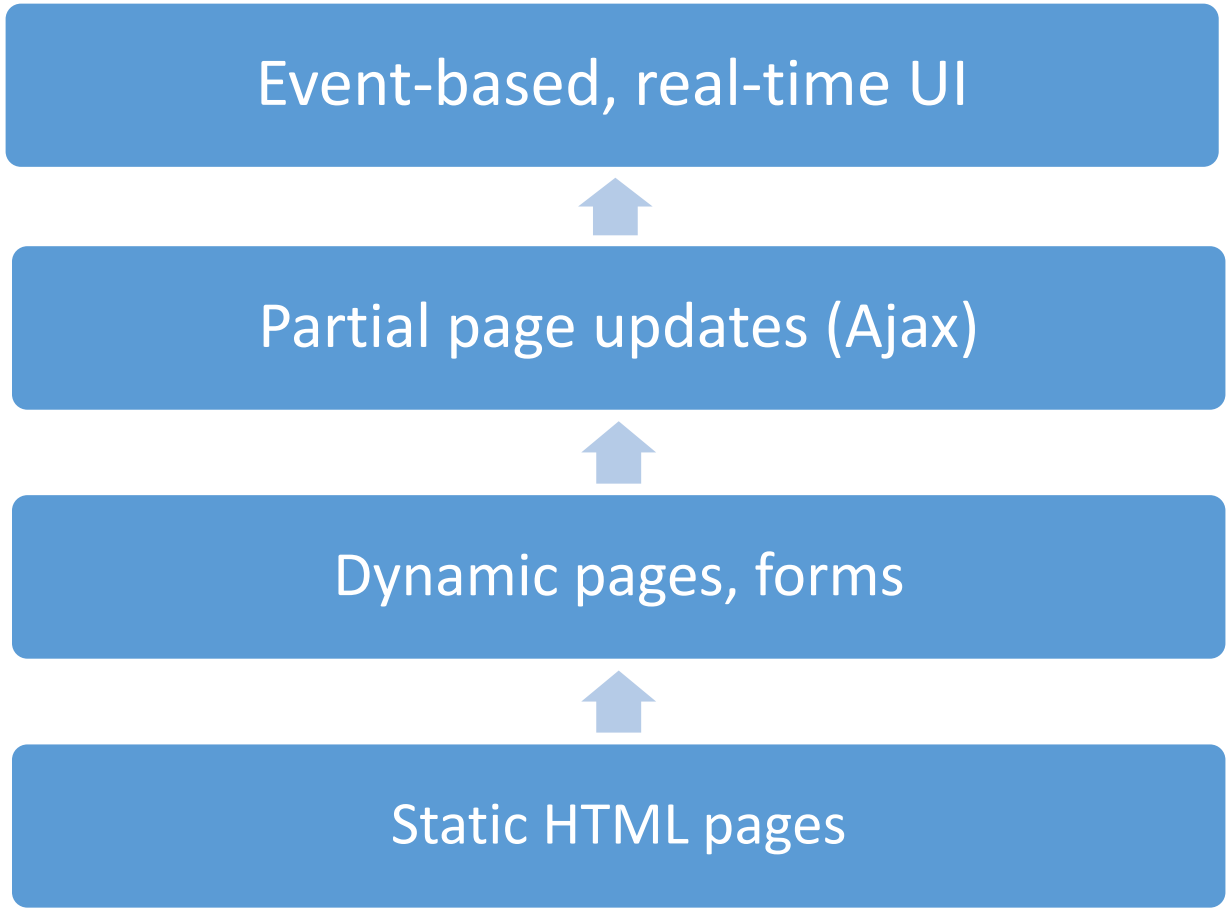
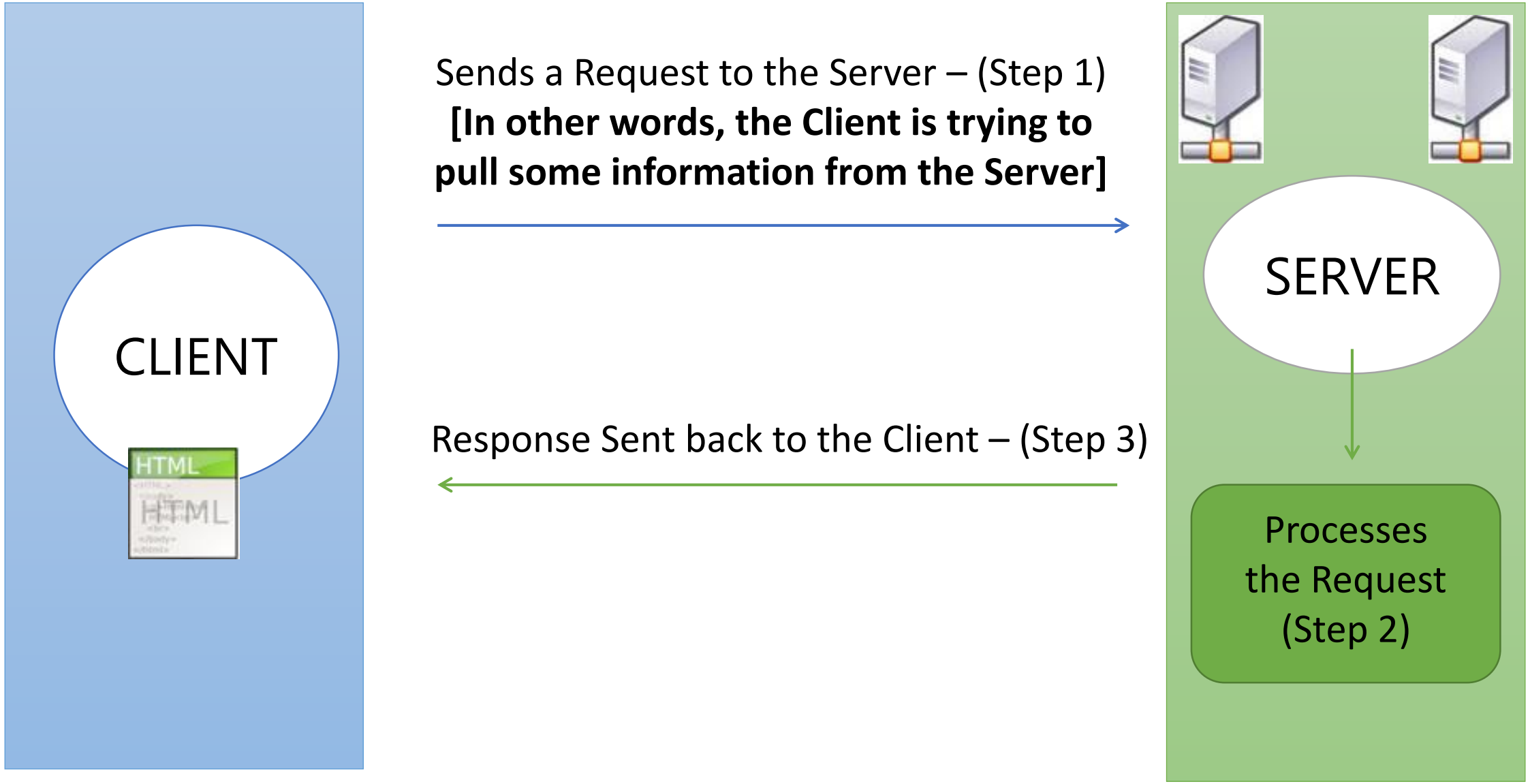
# ASP.Net Core SignalR

Welcome to the Real-time world of web

# Web Evolution



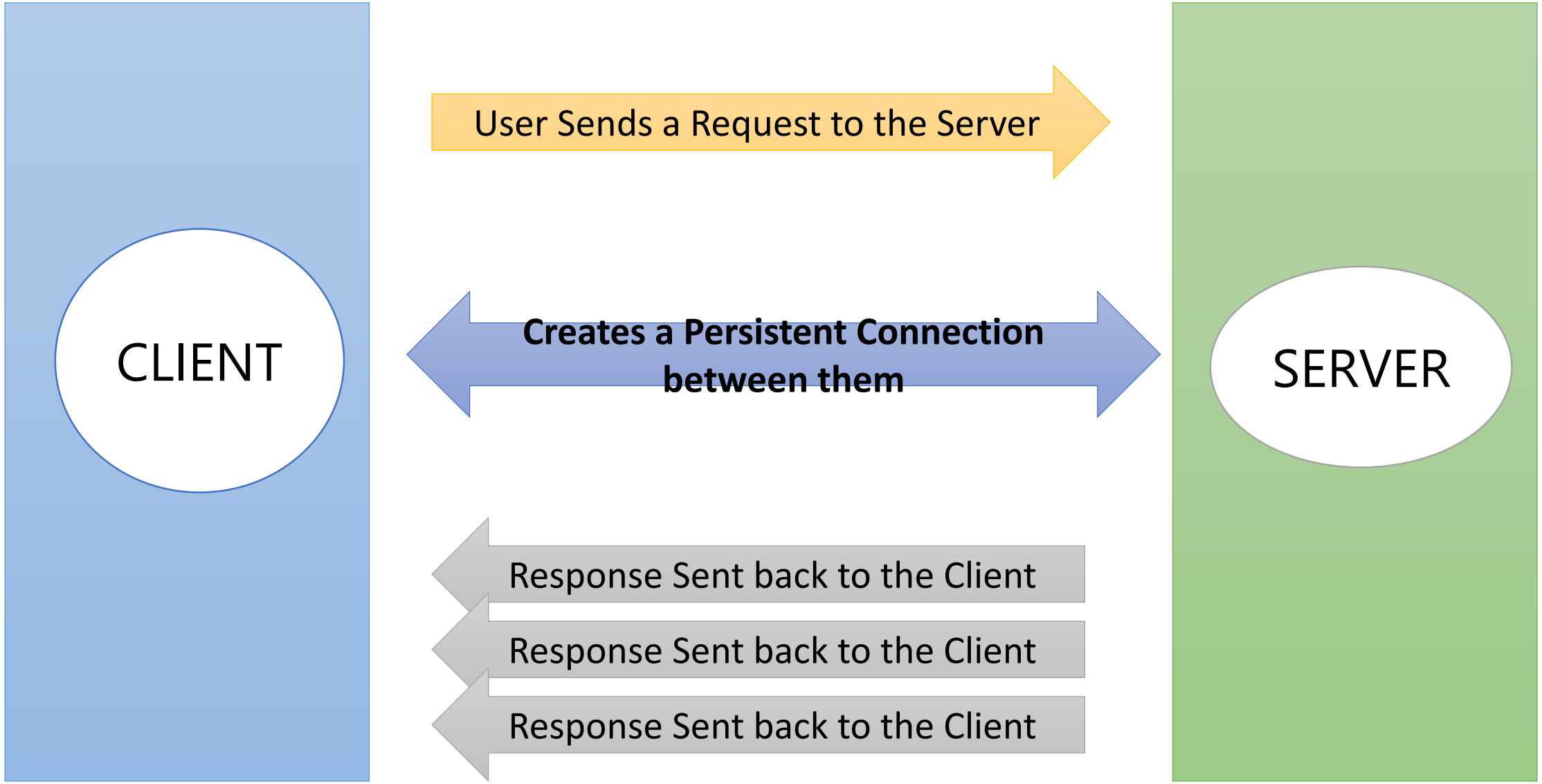
# Traditional Web Approach

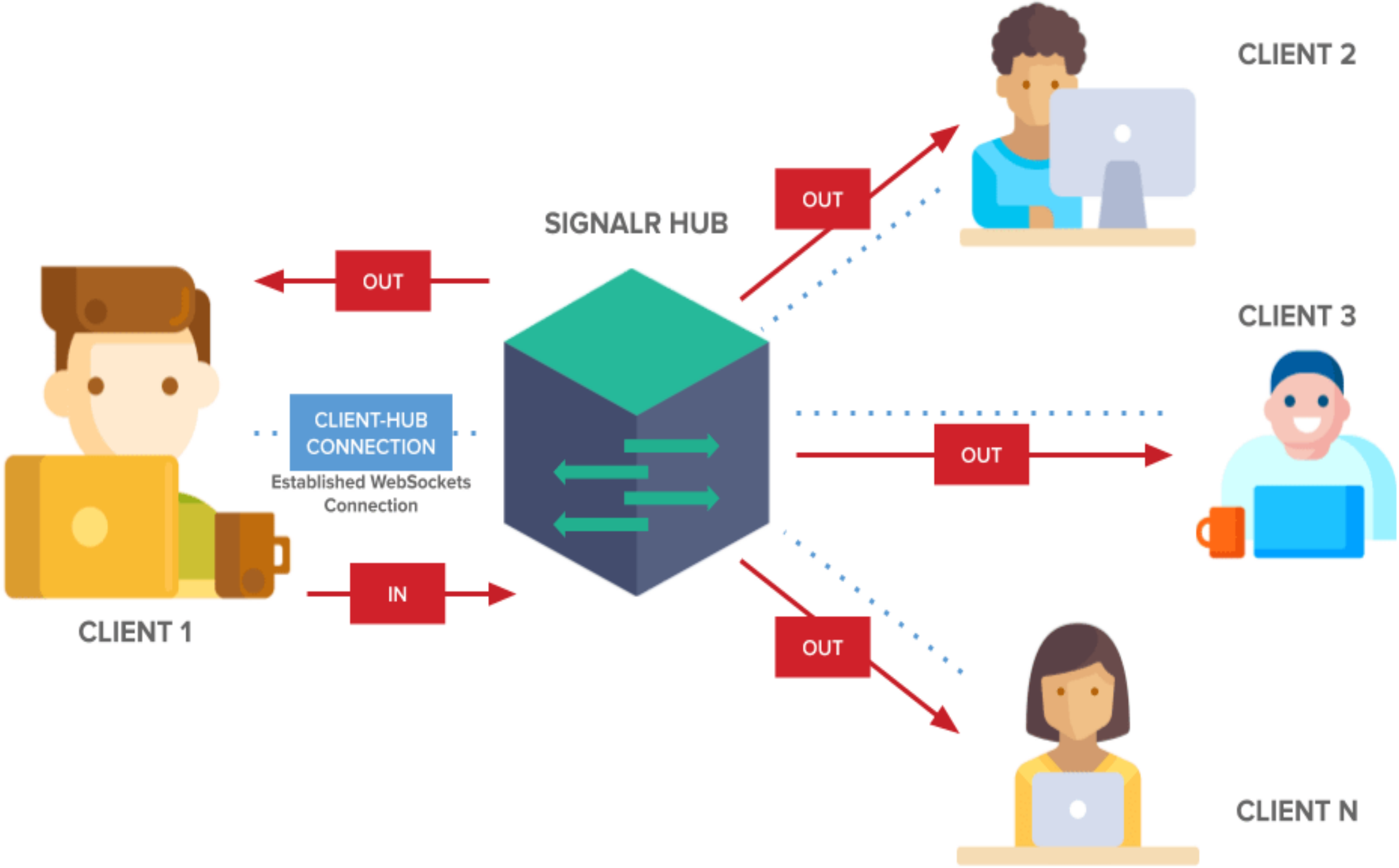


# What is Real Time Web Application?

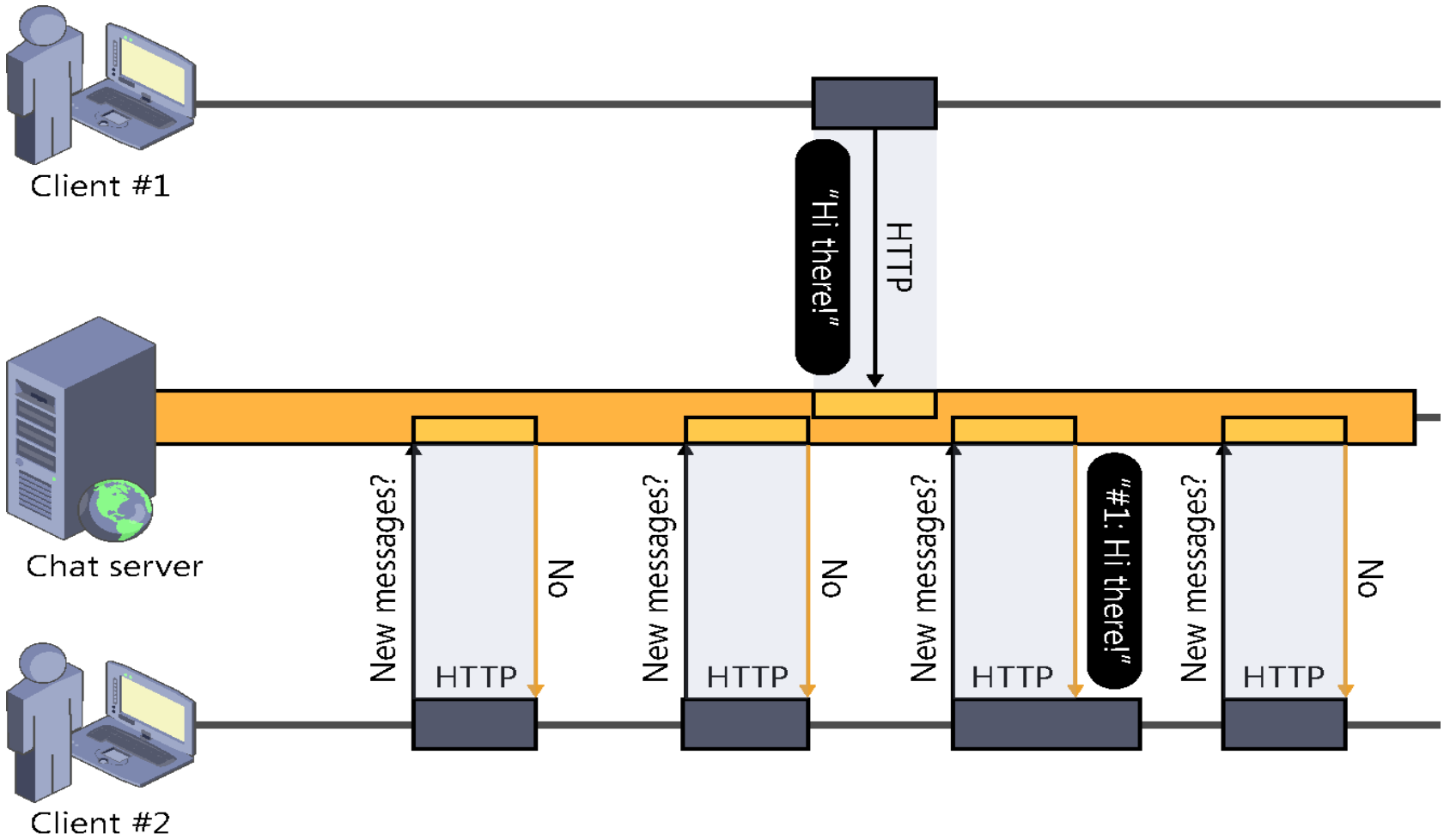
- In simple terms, “Real Time” means an immediate response being sent by the Server to the Client.
- Real Time is all about “Pushing” instead of “Pulling”
- Push Technology is completely different from Pull Technology. Its about getting told what’s new, instead of asking for what’s new!!!
- Facebook, Twitter, Yahoo Cricket Live, Stock Ticker

# Real Time Web Approach

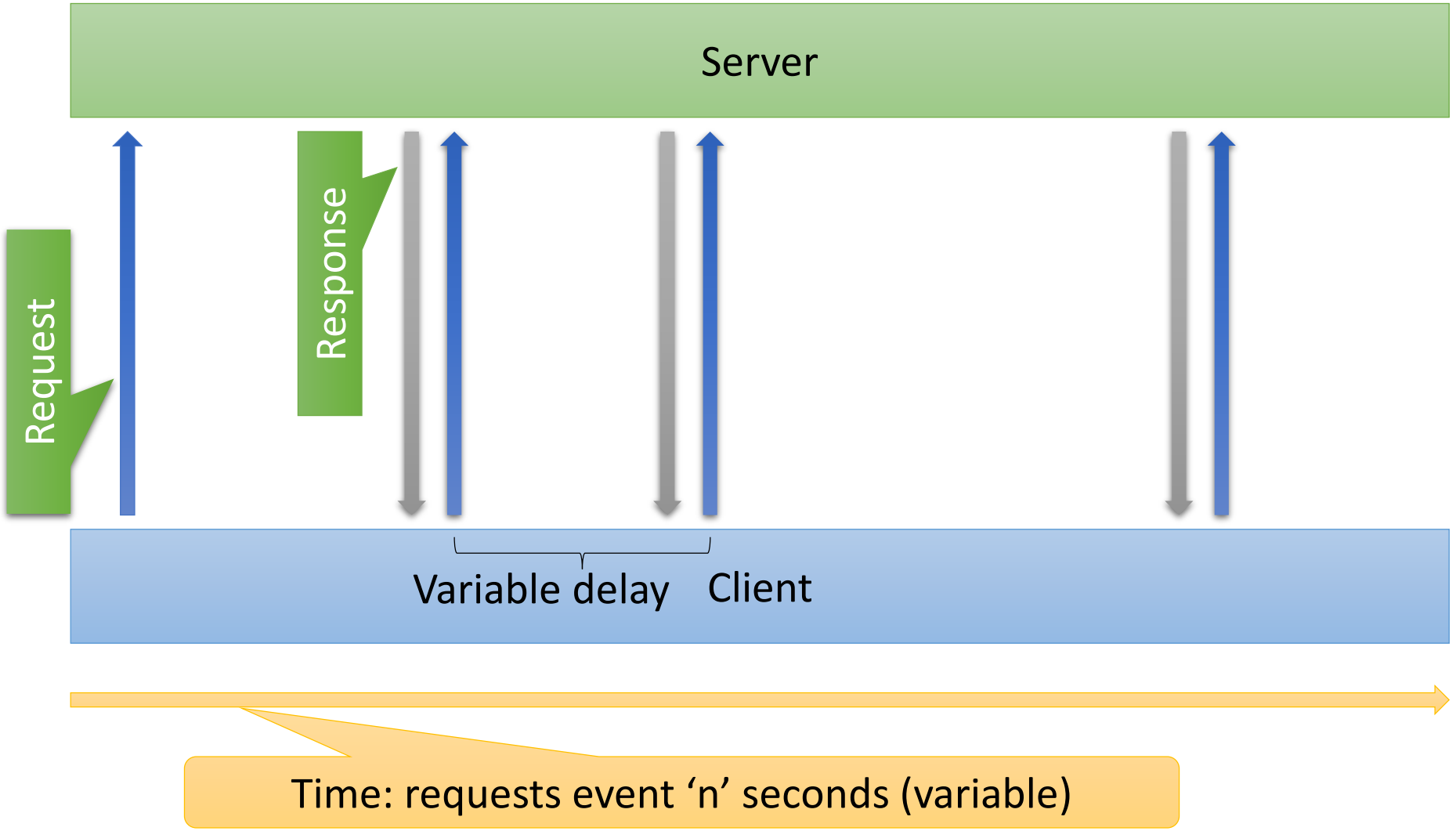



CLIENT 2
SIGNALR HUB
OUT
OUT
CLIENT 3
CLIENT-HUB
CONNECTION
Established WebSockets
Connection
OUT
IN
OUT
CLIENT 1
CLIENT N

# Polling


Client #1
Chat server
Client #2
"Hi there!"
HTTP
New messages?
HTTP
No
New messages?
HTTP
No
New messages?
HTTP
"#1: Hi there!"
New messages?
HTTP
No

# Long Polling



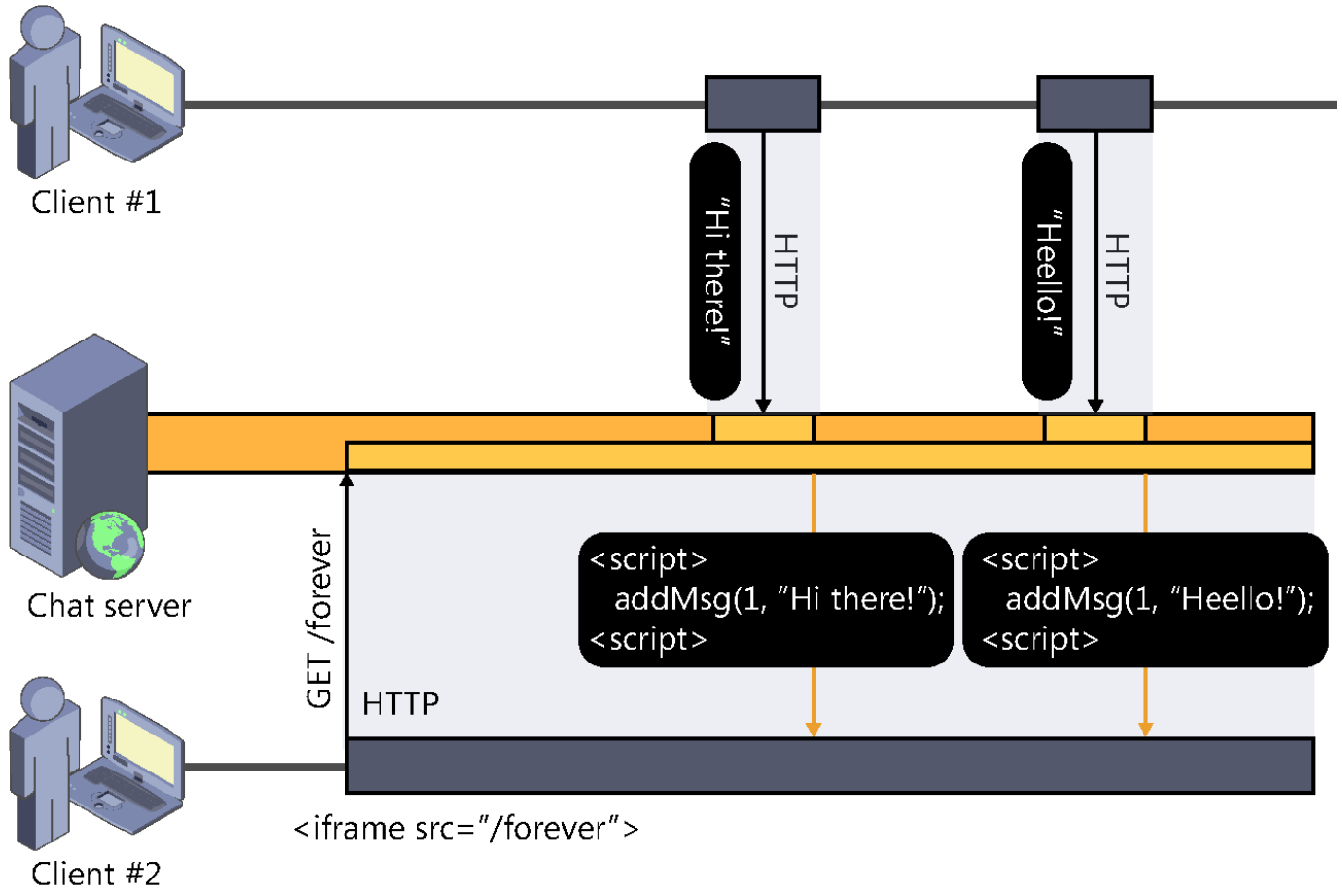
# Forever Frames

- Data is sent out in chunks.
- Chunked Encoding : is the feature in the HTTP 1.1 specification allowing a server to start sending a response before knowing its total length

- A hidden iframe element is opened in the browser after page load, establishing a long-lived connection inside the hidden iframe..

| Pros | Cons |
| --- | --- |
| Supported on IE Browser. | ▪ Iframes are loaded again and again with chunks of data.<br>▪ All script tags remain on the page.<br>▪ one-way realtime connection from server to client |
| | |

# Forever Frames

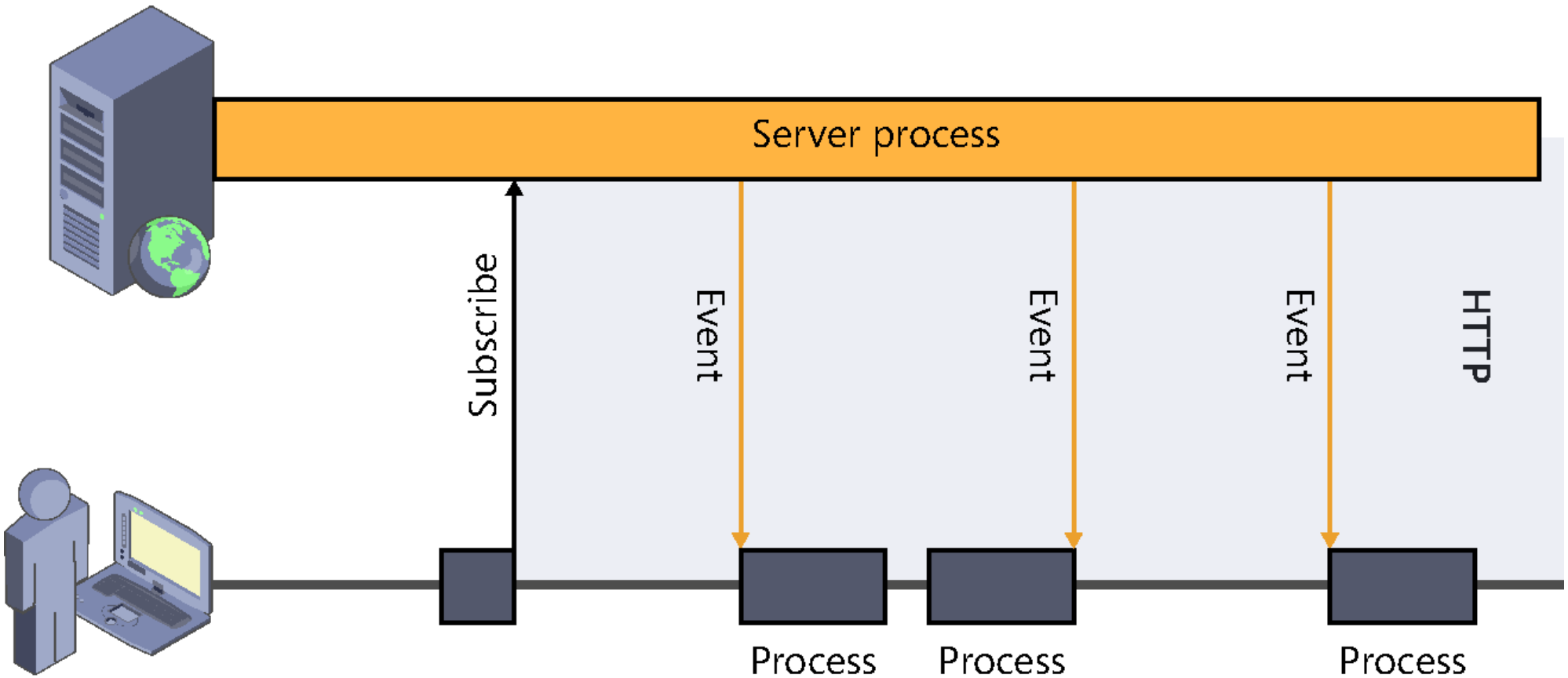


# Server Sent Events

- Requires a single connection between Client-Server.
- Uses Javascript API – “EventSource” through which Client can request a particular URL to receive data stream.
- Used to send Message Notifications or Continuous Data Streams.

| Pros | Cons |
| --- | --- |
| No need to reconnect | Works in server-to-client direction only |
| | Not supported in IE |

# Server Sent Events(SSE)



# Server Sent Events(SSE)

```
<script>
    var source = new EventSource('/elshafei/getevents');

    source.onmessage = function (event) {
        alert(event.data);
    };
</script>
```

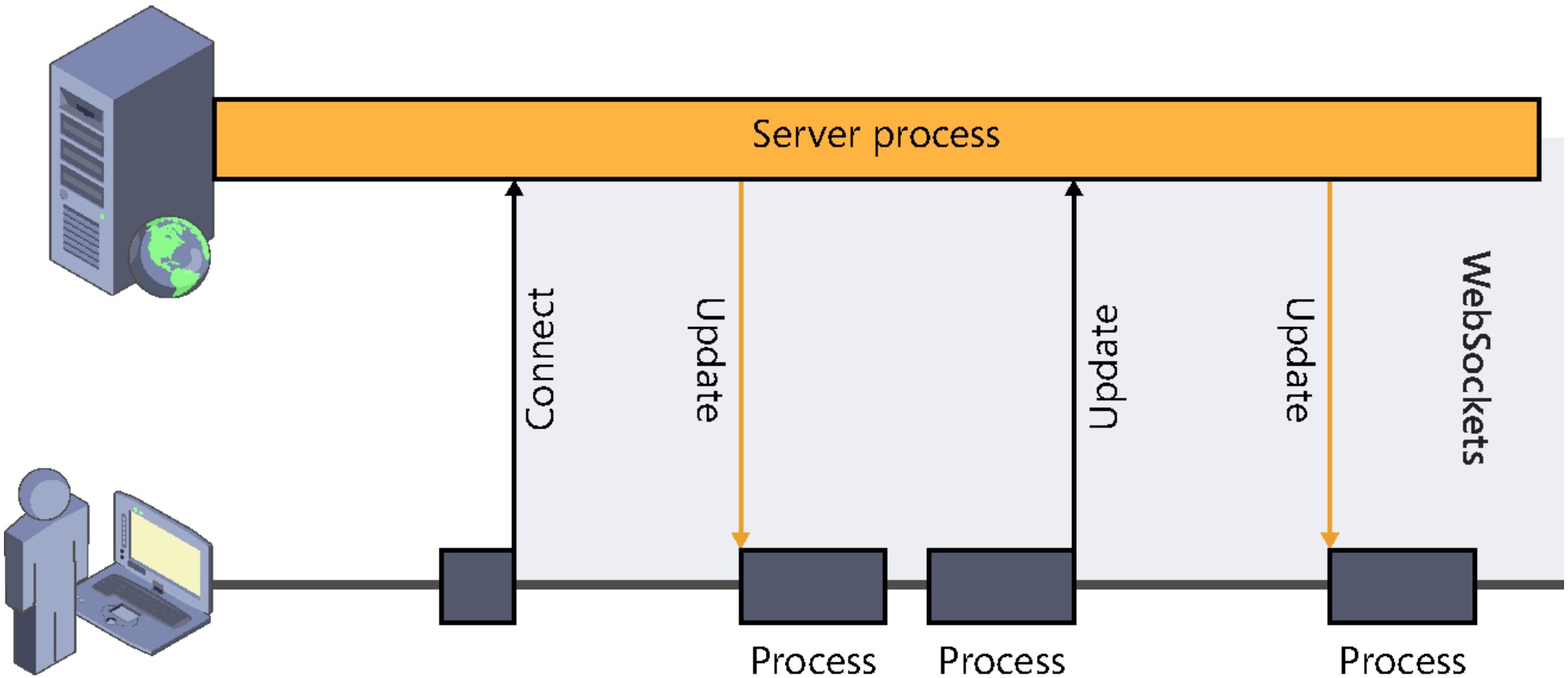
| Events | Description |
| --- | --- |
| onopen | When a connection to the server is opened |
| onmessage | When a message is received |
| onerror | When an error occurs |

# WebSocket

- A new transport technique which came up with HTML5.
- a full-duplex single socket connection over which messages can be sent between client and server
- It internally works on top of TCP protocol.

| Pros | Cons |
| --- | --- |
| Full-duplex persistent connection (both ways) | Supported only on latest browsers – (IE 10)<br>Windows 8, Windows Server 2012 or later |
| Fastest solution | Works only with IIS-8.0<br>.Net Framework 4.5+ |

# WebSocket


Server process
Connect
Update
Update
Update
WebSockets
Process
Process
Process

# WebSocket

```
<script>
    var ws = new WebSocket("ws://localhost:9998/index");
    ws.onopen = function () {
        // Web Socket is connected, send data using send()
        ws.send("Message to send");
        alert("Message is sent...");
    };
    ws.onmessage = function (evt) {
        var received_msg = evt.data;
        alert("Message is received...");
    };
    ws.onclose = function () {
        // WebSocket is closed.
        alert("Connection is closed...");
    };
</script>
```

# SignalR – Modern Web

- SignalR is an open-source library that simplifies adding real-time web functionality to apps.
- Concept intiated by “David Fowler” and “Damien Edwards”
- a server-side framework to write push services
- a set of client libraries to make push service communication easy to use on any platform
- optimized for asynchronous processing
- Open Source available on Github!!!

# Good candidates for SignalR

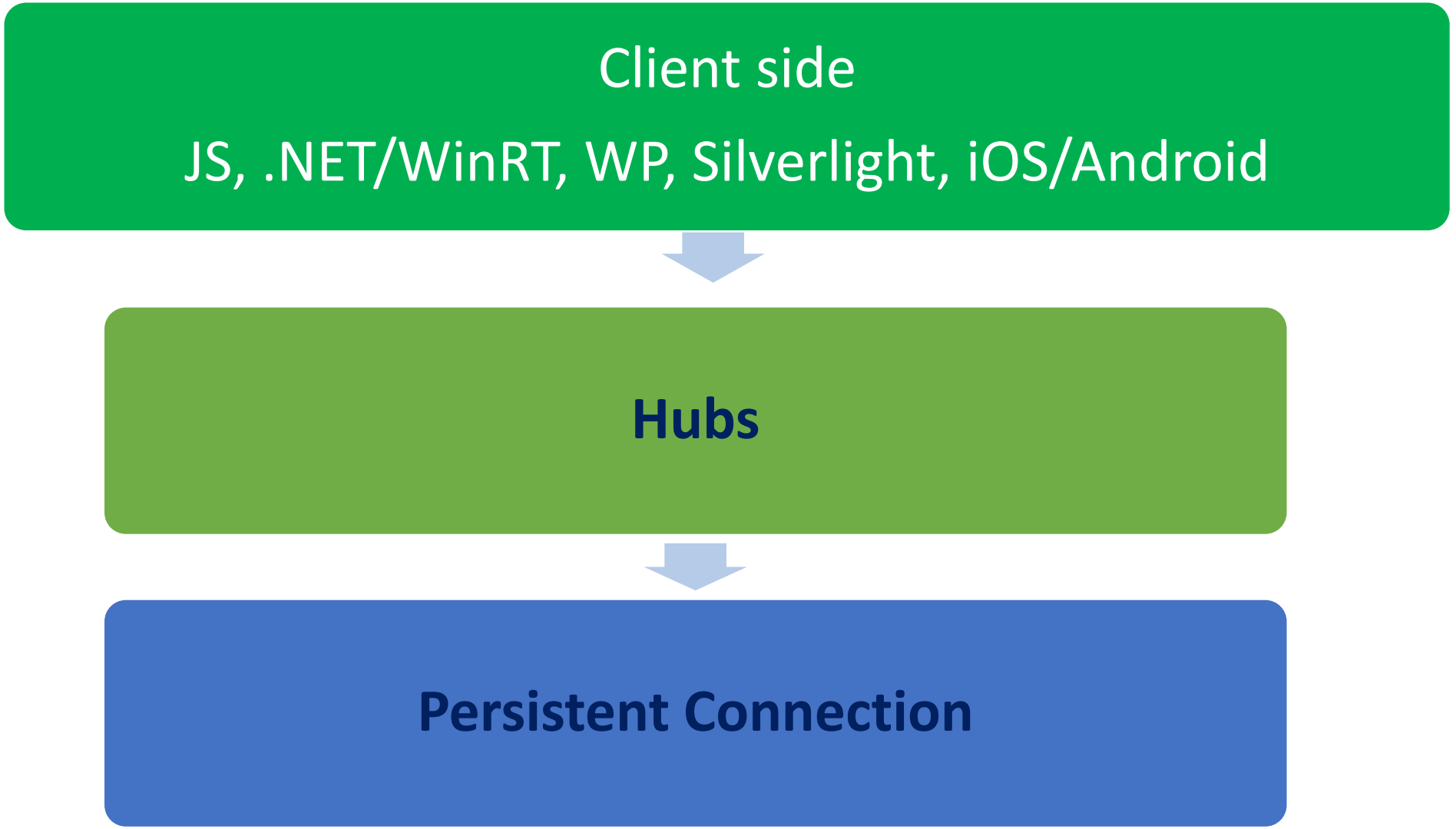
- Apps that require high frequency updates from the server. Examples are gaming, social networks, voting, auction, maps, and GPS apps.
- Dashboards and monitoring apps. Examples include company dashboards, instant sales updates, or travel alerts.
- Collaborative apps. Whiteboard apps and team meeting software are examples of collaborative apps.
- Apps that require notifications. Social networks, email, chat, games, travel alerts, and many other apps use notifications.

# SignalR

- **SignalR** handles connection management automatically, and lets you broadcast messages to all connected clients simultaneously, like a chat room. You can also send messages to specific clients. The connection between the client and server is persistent, unlike a classic HTTP connection, which is re-established for each communication.

- **SignalR** supports "server push" functionality, in which server code can call out to client code in the browser using Remote Procedure Calls (RPC), rather than the request-response model common on the web today.

# SignalR Connections



# ASP.NET Core SignalR

- **ASP.NET Core SignalR** is an open-source library that simplifies adding real-time web functionality to apps. Real-time web functionality enables server-side code to push content to clients instantly.

- SignalR provides an API for creating server-to-client ***remote procedure calls (RPC).*** The RPCs call JavaScript functions on clients from server-side .NET Core code.

# Transports

**SignalR** supports the following techniques for handling real-time communication (in order of graceful fallback):

- WebSockets
- Server-Sent Events
- Long Polling

**SignalR** automatically chooses the best transport method that is within the capabilities of the server and client.

# Configure SignalR(startup class)

- **ConfigureServices** :

  services.AddSignalR();

- **Configure:**

  app.MapHub<ChatHub>("/chatHub");

# Create a SignalR hub

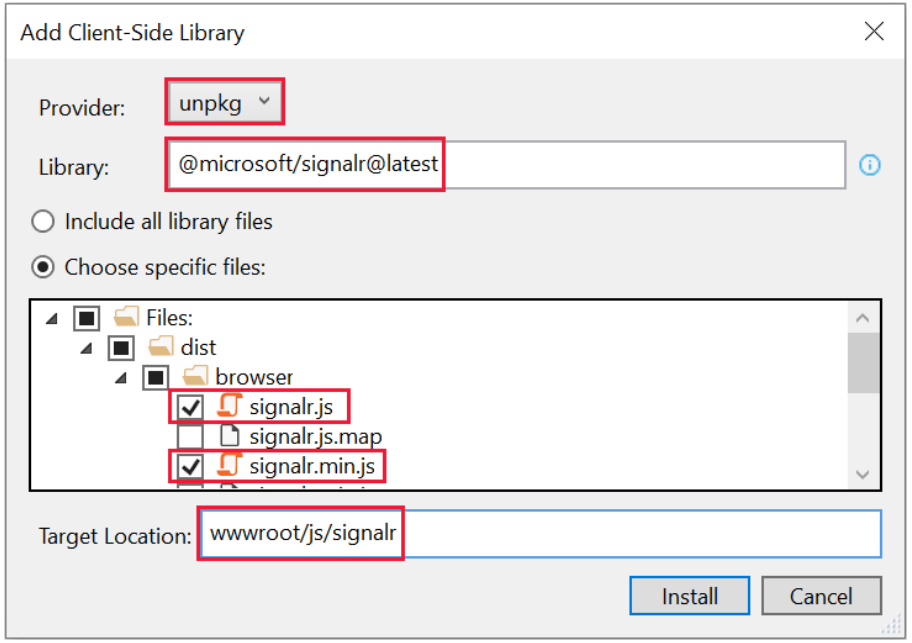
- In the *SignalRChat* project folder, create a Hubs folder.
- In the Hubs folder, create a *ChatHub.cs*

```
using Microsoft.AspNetCore.SignalR;
using System.Threading.Tasks;

namespace SignalRChat.Hubs
{
    public class ChatHub : Hub
    {
        public async Task SendMessage(string user, string message)
        {
            await Clients.All.SendAsync("ReceiveMessage", user, message);
        }
    }
}
```

# Add the SignalR client library

- In **Solution Explorer**, right-click the project, and select **Add** > **Client-Side Library**.




- **Use a Content Delivery Network (CDN)**

```
<script src="https://cdnjs.cloudflare.com/ajax/libs/microsoft-signalr/6.0.1/signalr.js"></script>
```

# Add SignalR client code

```
<script src="~/js/signalr/dist/browser/signalr.js"></script>
<script>
    //define connection
    var connection = new signalR.HubConnectionBuilder()
        .withUrl("/chatHub").build();

    //start connection
    connection.start()

    //define callback fun
    connection.on("ReceiveMessage", function (user, message) {
        //anycode
    });

    //call server method
    connection.invoke("SendMessage", user, message)

</script>
```

# Invoke vs send

## **invoke()** → ***Request / Response***

- Calls a server method and waits for a result
- Returns a Promise
- Supports async/await and error handling
- Use for business logic or operations requiring confirmation

```
const result = await connection.invoke("Add", 5, 3);
console.log(result); // 8
```

## **send()** → ***Fire & Forget***

- Calls a server method without waiting for a response
- Does not return a result
- No built-in error handling
- Use for events / notifications

```
connection.send("SendMessage", "Mohamed", "Hello");
```

# Configure Transport Type

If you don’t specify a transport, SignalR will choose automatically:

Order: **WebSockets → SSE → Long Polling**

**Force a Specific Transport**

- You can explicitly choose which transport to use when creating the connection:

```
var con = new
signalR.HubConnectionBuilder().withUrl("/chat",signalR.HttpTransportType.ServerSentEvents).build();
```

**Available Transport Options**

- signalR.HttpTransportType.WebSockets
- signalR.HttpTransportType.ServerSentEvents
- signalR.HttpTransportType.LongPolling

# Protocol-Based Messaging

- SignalR does **not call methods directly**
  It uses **protocol-based messages** over a persistent connection
- Communication = **Serialized Messages (RPC style)**

## How It Works

1-Client calls:

```
connection.invoke("SendMessage", "Mohamed", "Hello");
```

2- Converted to message:

```
{
"type": 1,
"target": "SendMessage",
 "arguments": ["Mohamed", "Hello"],
 "invocationId": "1"
}
```

3-Sent via transport (WebSocket / SSE / Long Polling)

4- Server:

Matches target → Hub method

Binds arguments → executes method

# JavaScript Client Logging

Configure logging when creating the connection:

**What You Can See in Logs**

Connection start / stop ,Transport selection (WebSockets / fallback), Messages sent via invoke / send , Server responses ,Reconnect events and failures

**Logging Levels**

- Trace → Full protocol details
- Debug → Development-level details
- Information → Connection lifecycle + key events
- Warning → Non-fatal issues
- Error → Failures only
- None → Disable logging

```
const connection = new signalR.HubConnectionBuilder()
.withUrl("/chatHub")
.configureLogging(signalR.LogLevel.Information)
.build();
```

# The Clients object

| Property | Description |
|---|---|
| All | Calls a method on all connected clients |
| Caller | Calls a method on the client that invoked the hub method |
| Others | Calls a method on all connected clients except the client that invoked the method |
| AllExcept | Calls a method on all connected clients except for the specified connections |
| Client | Calls a method on a specific connected client |
| Clients | Calls a method on specific connected clients |
| Group | Calls a method on all connections in the specified group |
| GroupExcept | Calls a method on all connections in the specified group, except the specified connections |
| Groups | Calls a method on multiple groups of connections |
| OthersInGroup | Calls a method on a group of connections, excluding the client that invoked the hub method |
| User | Calls a method on all connections associated with a specific user |
| Users | Calls a method on all connections associated with the specified users |

# Context in SignalR

- Represents the current connection
- Contains identity and connection metadata
- Only available inside the Hub

**Context .ConnectionId:**

- A unique identifier for each client connection
- Generated by SignalR when the connection is established
- One user can have multiple ConnectionIds (multiple tabs / devices)

# Groups in SignalR

- A group is a collection of connections associated with a name.
- Logical channels to organize connections Used for scalable broadcasting
- Messages can be sent to all connections in a group.
- A connection can be a member of multiple groups.

- Connections are added to or removed from groups via the AddToGroupAsync and RemoveFromGroupAsync methods.

# ASP.NET Core SignalR .NET Client

- The *Microsoft.AspNetCore.SignalR.Client* package is required for .NET clients to connect to SignalR hubs.

```
HubConnection con;
1 reference
public Form1()
{
    InitializeComponent();
     con = new HubConnectionBuilder()
        .WithUrl("https://localhost:7269/chat").Build();

    con.StartAsync();
    con.On<string, string>("newmess", (n, m) => listBox1.Items.Add(n + m));
}

1 reference
private void button1_Click(object sender, EventArgs e)
{
    con.InvokeAsync("sendmessage", "ali", textBox1.Text);
}
```

# Handle lost connection (Automatically reconnect)

Re-establishing the connection after it drops (network issues, server restart)

- Without any parameters, WithAutomaticReconnect() configures the client to wait 0, 2, 10, and 30 seconds respectively before trying each reconnect attempt, stopping after four failed attempts.
- ConnectionId **changes** after reconnect
- You must **re:Join Groups ,Sync state/data** if needed

```
HubConnection connection= new HubConnectionBuilder()
    .WithUrl(new Uri("http://127.0.0.1:5000/chathub"))
    .WithAutomaticReconnect()
    .Build();
```

# Handle events for a connection

- The SignalR Hubs API provides the OnConnectedAsync and OnDisconnectedAsync virtual methods to manage and track connections

```
public override async Task OnConnectedAsync()
{
    await Groups.AddToGroupAsync(Context.ConnectionId, "SignalR Users");
    await base.OnConnectedAsync();
}
```

# Send messages from outside a hub

- The SignalR hub is the core abstraction for sending messages to clients connected to the SignalR server.
- It's also possible to send messages from other places in your app using the *IHubContext* service.
- in ASP.NET Core SignalR, you can access an instance of IHubContext via dependency injection. You can inject an instance of IHubContext into a controller, middleware, or other DI service

# Send messages from outside a hub

```
public class HomeController : Controller
{
    private readonly IHubContext<NotificationHub> _hubContext;

    public HomeController(IHubContext<NotificationHub> hubContext)
    {
        _hubContext = hubContext;
    }

    public async Task<IActionResult> Index()
    {
        await _hubContext.Clients.All.SendAsync("Notify", $"Home page loaded at: {DateTime.Now}");
        return View();
    }
```

# Cross-origin connections (CORS)

- CORS (Cross-Origin Resource Sharing) is a browser security mechanism that controls whether a web app running on one domain can communicate with a server on a different domain.
- SignalR uses HTTP (for handshake),WebSockets / SSE / Long Polling

All of these are subject to browser CORS rules when the client **is on a different origin.**

```
// UseCors must be called before MapHub.
app.UseCors();
```

```
builder.Services.AddCors(options => {
        options.AddDefaultPolicy( builder => {
        builder.WithOrigins("https://example.c
        om") .AllowAnyHeader() .
        WithMethods("GET", "POST")
        .AllowCredentials();
    });
});
```

# Change the name of a hub method

- By default, a server hub method name is the name of the .NET method. To change this default behavior for a specific method, use the *HubMethodName* attribute.
- The client should use this name instead of the .NET method name when invoking the method

```
[HubMethodName("SendMessageToUser")]
public async Task DirectMessage(string user, string message)
    => await Clients.User(user).SendAsync("ReceiveMessage", user, message);
```

# Strongly typed hubs

- A drawback of using SendAsync is that it relies on a string to specify the client method to be called. This leaves code open to runtime errors if the method name is misspelled or missing from the client

```
public interface IChatClient
{
    Task ReceiveMessage(string user, string message);
}
```

```
public class StronglyTypedChatHub : Hub<IChatClient>
{
    public async Task SendMessage(string user, string message)
        => await Clients.All.ReceiveMessage(user, message);

    public async Task SendMessageToCaller(string user, string message)
        => await Clients.Caller.ReceiveMessage(user, message);

    public async Task SendMessageToGroup(string user, string message)
        => await Clients.Group("SignalR Users").ReceiveMessage(user, message);
}
```

# Scalability in ASP.NET SignalR

- **Scalability:**The ability to handle **increasing users, connections, and messages** Without degrading performance or reliability
- **Limitation:** SignalR stores(Connections ,Groups)**In-memory (per server)**
- Each server has its own isolated state
- **Problem in Multi-Server**: Client A → Server 1 ,Client B → Server 2 when calling : Clients.All.SendAsync(...)

❌ Message is NOT shared across servers

❌ Some clients will miss updates

**Solution**: Backplane

# Scalability in ASP.NET SignalR

**Backplane:** A shared messaging layer between servers **Synchronizes** (**Messages ,Connection events**)

Scaling Options:

**1) Redis Backplane (Redis)**

Fast, self-hosted but Requires infrastructure management

```
builder.Services.AddSignalR()
.AddStackExchangeRedis("connection_string");
```

**2) Azure SignalR Service (Microsoft)**

Fully managed, Handles massive scale but Paid service